Brazo XK-JS1-1 y su gemelo digital

Guía completa: planta, programas del PLC, modelado y software

ETN-902 / ETN-1034 / ETN-1000 · UMSA · II-2026

10 de noviembre de 2026

Resumen

Esta guía describe por completo el brazo XK-JS1-1 del laboratorio y el gemelo digital construido sobre él: qué hace el brazo, cómo está formado, cómo se conectan sus partes, qué hacen los dos programas del PLC (explicados con diagramas ladder), cómo se modeló su cinemática, cómo se comunica la PC con el PLC y cómo está escrito el software (la app de Python y la página web). Los manuales de Linux y de Windows explican cómo instalar y usar la app; el documento de arquitectura profundiza en el protocolo PPI y en las páginas web. Los datos no medidos en la planta se marcan como **SUPUESTO**.

Índice

I La planta **3**

1. Qué hace el brazo **3**

2. Componentes y cómo se conectan **4**
2.1. Motores, drivers y transmisión . . . 4
2.2. Mapa de entradas y salidas . . . 5
2.3. La limitación de cableado: un solo tren de pulsos para X e Y . . . 5
2.4. El panel . . . 5

II Los programas del PLC **6**

3. Conceptos del S7-200 que usan los programas **6**
3.1. El ciclo del PLC . . . 6
3.2. Cómo se lee un diagrama ladder . . . 6
3.3. Salidas de pulsos (PTO) . . . 6
3.4. Máquinas de pasos . . . 7

4. Programa original de secuencias **7**
4.1. Arranque y órdenes (OB1, segmento 1) . . . 7
4.2. Modo manual (SBR20) . . . 8
4.3. Homing (SBR0) . . . 8
4.4. Secuencias automáticas (SBR21) . . . 9
4.5. Estado para la PC y limitaciones . . . 9

5. Programa del gemelo: `brazo_gemelo` 10
5.1. Cuenta de pulsos . . . 10
5.2. El buzón de órdenes . . . 11
5.3. Ejecución de un movimiento . . . 12
5.4. Watchdog y paro . . . 13
5.5. Comparación de los dos programas . . . 13

III El modelado 13

6. Levantamiento de medidas 13

7. Cinemática 13
7.1. El mecanismo . . . 13
7.2. Cinemática directa: de los carros a la garra . . . 14
7.3. Cinemática inversa: de la garra a los carros . . . 14
7.4. Jacobiano y singularidades . . . 15
7.5. Espacio de trabajo . . . 15
7.6. Conversión a pulsos . . . 15

8. Modelo 3D 16

IV El software 16

9. Arquitectura 16

10. La app: modos y cambio de modo 17

11. Comunicación con el PLC 17
11.1. El puerto serie en dos sistemas . . . 17
11.2. El protocolo PPI . . . 17
11.3. Perfiles: la app se adapta al programa del PLC . . . 18
11.4. El estimador (programa original) . . . 18

12. La página y los modos de mando 19

13. Seguridad por capas 19

V Referencia 19

14. Archivos del proyecto 19

15. Glosario 20

Parte I
La planta

1. Qué hace el brazo

El XK-JS1-1 es un brazo didáctico de **4 grados de libertad** que toma piezas pequeñas y las deja en otro lugar. Sus cuatro movimientos son:

Movimiento	Qué hace	Accionamiento
Base (θ)	gira todo el brazo alrededor de un eje vertical	motor paso a paso + engranaje
Carro X (s_X)	desplaza el brazo hacia adelante o atrás: cambia el *alcance*	motor paso a paso + husillo inferior
Carro Y (s_Y)	mueve un bloque que, por una biela, levanta o baja el brazo: cambia la *altura*	motor paso a paso + husillo superior
Garra	abre o cierra las mordazas	salida Q0.7 del PLC (1 = cerrada)

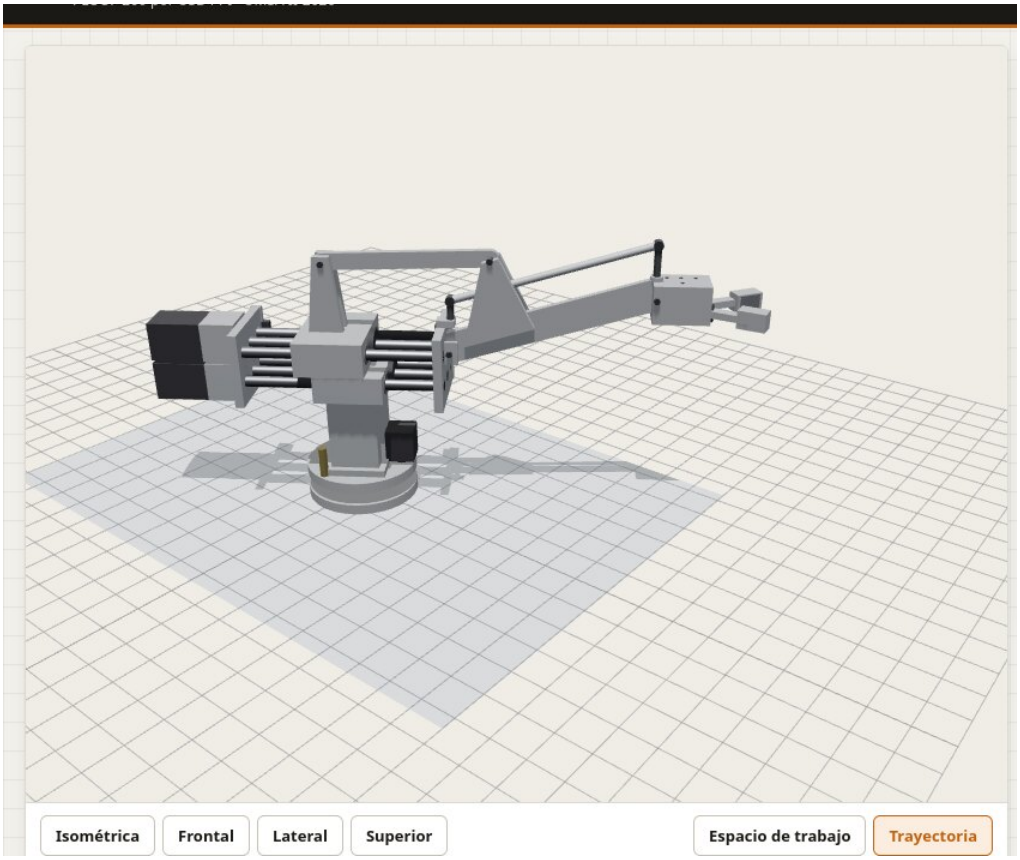


Figura 1: El brazo dibujado por el gemelo digital con las mismas cotas que la máquina real ($\theta = 35°$, $s_X = 70$ mm, $s_Y = 95$ mm).

En la planta original, el brazo solo podía moverse con los pulsadores del panel (modo manual) o repetir dos rutinas grabadas en el PLC (modo automático). Con el gemelo digital, además, se ve en 3D en tiempo real, se lleva a cualquier punto y se manda desde la PC o desde celulares.

2. Componentes y cómo se conectan

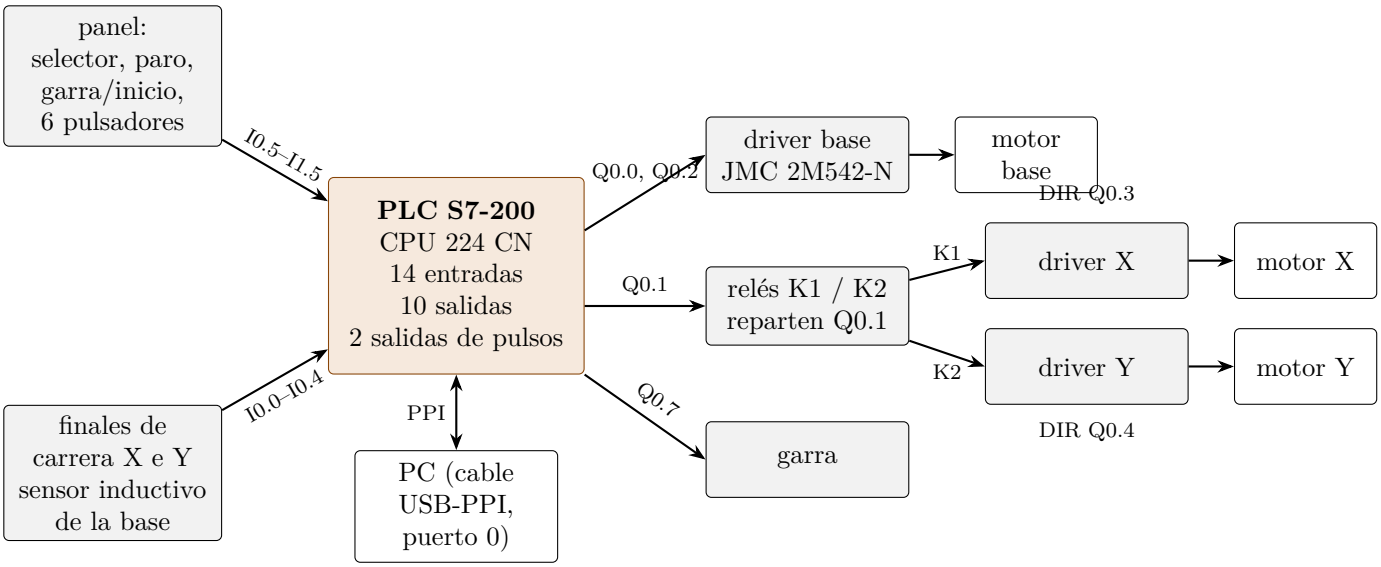


Figura 2: Diagrama de bloques de la planta.

2.1. Motores, drivers y transmisión

Los tres ejes usan motores paso a paso NEMA 17 (200 pasos por vuelta) con drivers JMC 2M542-N. Cada driver recibe dos señales del PLC: **PUL** (un pulso = un micropaso) y **DIR** (sentido). La configuración de los interruptores DIP se leyó del tablero:

Interruptores	Posición	Significado	Efecto
SW1–SW3	OFF OFF ON	corriente	2,37 A pico
SW4	OFF	corriente en reposo	la mitad al detenerse (menos calor)
SW5–SW8	ON ON ON OFF	micropasos	1000 pulsos por vuelta

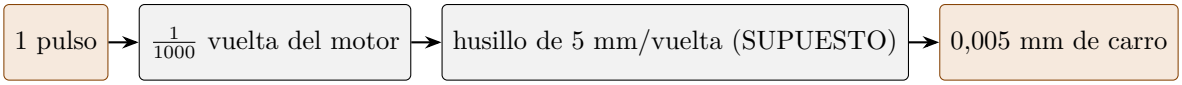


Figura 3: Cadena de transmisión de los carros X e Y: 200 pulsos por milímetro.

- **Carros X e Y**: $k = 1000/5 = 200$ pulsos/mm. Recorrido útil 0–170 mm (0–34 000 pulsos) entre finales de carrera.
- **Base**: la reducción no se conocía (el manual dice «engranaje»). Se calibró midiendo un giro con transportador: **190,3 pulsos por grado**. El sentido (Q0.2 = 0 positivo) es SUPUESTO.
- **Velocidad**: a 4 kHz los carros avanzan 4000/200 = 20 mm/s. Las salidas de pulsos llegan hasta 20 kHz, pero los programas no tienen rampas de aceleración, así que conviene no pasar de 4–5 kHz para no perder pasos.

2.2. Mapa de entradas y salidas

Entrada	Función	Salida	Función
I0.0	sensor inductivo de la base ($\theta = 0$)	Q0.0	pulsos de la base (PTO0)
I0.1	final de carrera X atrás (origen)	Q0.1	pulsos de X e Y (PTO1), compartidos
I0.2	final de carrera X adelante	Q0.2	DIR base (1 = hacia el inductivo)
I0.3	final de carrera Y abajo (origen)	Q0.3	DIR X (0 = adelante, 1 = atrás)
I0.4	final de carrera Y arriba	Q0.4	DIR Y (0 = arriba, 1 = abajo)
I0.5	selector: 0 = manual, 1 = automático/remoto	Q0.5	relé K1: Q0.1 al driver X
I0.6	pulsador de paro	Q0.6	relé K2: Q0.1 al driver Y
I0.7	garra / inicio	Q0.7	garra (1 = cerrada)
I1.0, I1.1	jog base + / −		
I1.2, I1.3	jog X adelante / atrás		
I1.4, I1.5	jog Y arriba / abajo		

2.3. La limitación de cableado: un solo tren de pulsos para X e Y

La CPU 224 tiene solo dos salidas de pulsos rápidas (Q0.0 y Q0.1). Q0.0 mueve la base; **Q0.1 mueve los dos carros**, y los relés K1 y K2 eligen a qué driver llega. Consecuencias:

- X e Y pueden moverse a la vez, pero **a la misma velocidad** y recibiendo los mismos pulsos;
- para un movimiento con recorridos distintos hay que dividirlo en un *tramo común* (K1 y K2 cerrados) y un *resto* (solo el relé del eje más largo);
- la garra no puede describir una recta en diagonal.

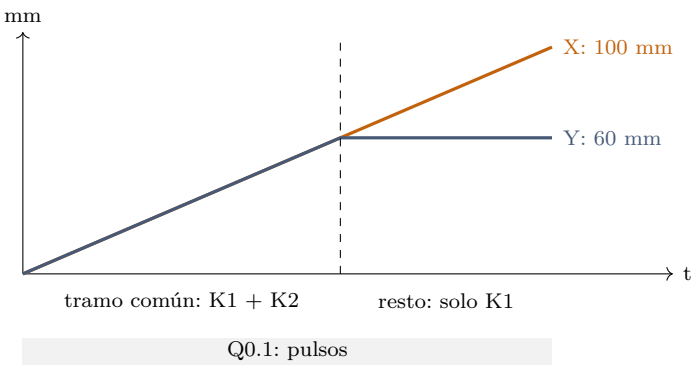


Figura 4: Movimiento de X 100 mm e Y 60 mm con el tren de pulsos compartido.

2.4. El panel

- **Selector I0.5**: en *manual* el brazo obedece a los pulsadores; en *automático* ejecuta el homing y las secuencias (programa original) o las órdenes de la PC (programa del gemelo).
- **Paro I0.6**: detiene los trenes de pulsos.
- **I0.7**: en manual abre/cierra la garra; en automático inicia (homing o secuencia).

- **Pulsadores I1.0–I1.5**: mueven cada eje en los dos sentidos mientras se mantienen apretados (jog).

Atención. La seta de emergencia del tablero corta la alimentación de potencia; es la protección principal. Ningún programa la reemplaza.

Parte II
Los programas del PLC

3. Conceptos del S7-200 que usan los programas

3.1. El ciclo del PLC

El PLC repite sin parar un *ciclo*: lee todas las entradas, ejecuta el programa de arriba abajo (el bloque principal OB1 y las subrutinas que llama) y escribe todas las salidas. Un ciclo dura unos pocos milisegundos.

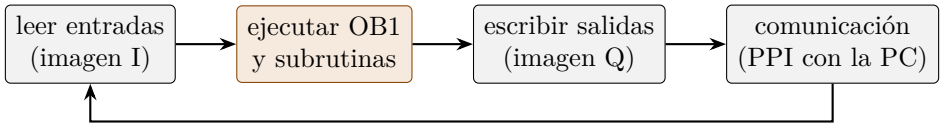
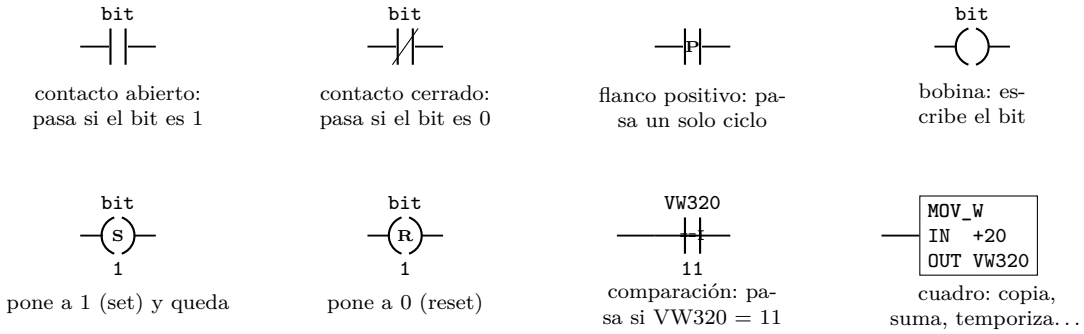


Figura 5: Ciclo del PLC. Las peticiones de la PC se atienden entre ciclos.

3.2. Cómo se lee un diagrama ladder

Un programa ladder se dibuja como un circuito eléctrico: a la izquierda una barra con «corriente»; cada *segmento* (red) lleva la corriente por *contactos* hasta *bobinas* o *cuadros*. Si el camino está cerrado, la bobina se activa o el cuadro se ejecuta.



Los programas se escriben y guardan en *AWL* (lista de instrucciones); Micro/WIN los muestra en ladder. Equivalencias: `LD` empieza un camino, `A` = contacto en serie, `O` = en paralelo, `N` = negado, `EU` = flanco positivo, `=` = bobina, `S`/`R` = set/reset.

3.3. Salidas de pulsos (PTO)

Q0.0 y Q0.1 pueden generar trenes de pulsos por hardware, sin depender del ciclo. Se configuran con bytes especiales y se arrancan con la instrucción `PLS`:

PTO0 (Q0.0)	PTO1 (Q0.1)	Contenido
SMB67	SMB77	control: `16#85` = activar PTO, tiempo en µs, un solo segmento, actualizar ciclo y cantidad
SMW68	SMW78	ciclo del pulso en µs (250 µs = 4 kHz)
SMD72	SMD82	cantidad de pulsos
SM66.7	SM76.7	1 = PTO en reposo (terminó)

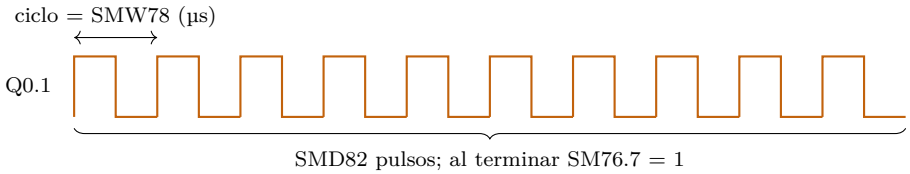


Figura 6: Un tren de pulsos: el driver avanza un micropaso por pulso.

Para detener un PTO antes de tiempo se escribe `SMB77 = 0` y se ejecuta `PLS 1`.

3.4. Máquinas de pasos

Las secuencias se programan como **máquinas de pasos**: una palabra de memoria (por ejemplo VW320) guarda el número de paso; cada segmento comprueba «¿estoy en el paso N?», hace su tarea y, cuando se cumple la condición, escribe el número del paso siguiente. Es el equivalente en ladder de un diagrama de estados.

4. Programa original de secuencias

Es el programa que tenía el PLC al empezar el proyecto (respaldado el 26/10/2026 en `respaldo_vm/programa_plc` exportado como `proyecto1_abierto.awl`). Es una versión del programa de Miguel con un pequeño buzón (VB104) para recibir órdenes de la PC.

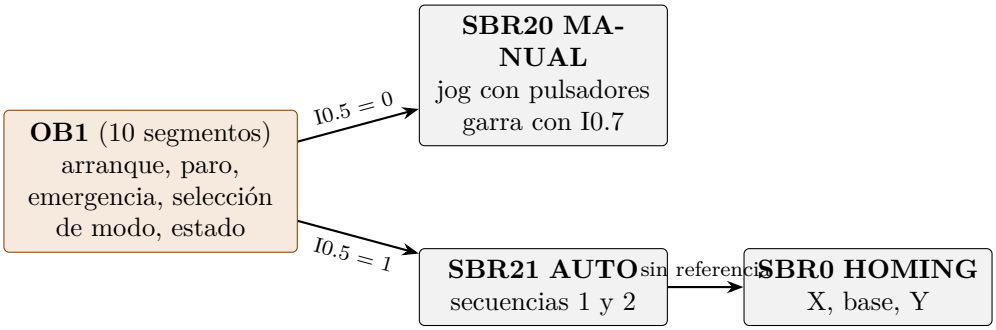


Figura 7: Estructura del programa original.

4.1. Arranque y órdenes (OB1, segmento 1)

El inicio puede venir del pulsador I0.7 o de la PC (V104.0), siempre que no haya emergencia (V104.2) ni paro remoto (V104.6). Su flanco (M5.0) arranca el homing o la siguiente secuencia.

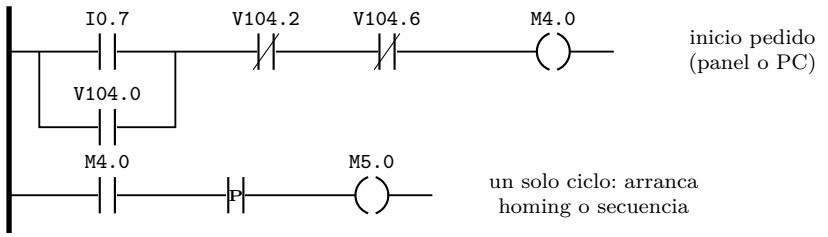


Figura 8: Inicio: I0.7 *o* V104.0, y sin emergencia ni paro remoto.

El mismo segmento implementa un **latido**: si V104.7 (que la PC alterna) no cambia en 5 s, el temporizador T61 borra VB104, de modo que una PC colgada no deja órdenes pendientes.

4.2. Modo manual (SBR20)

Cada pulsador de jog da paquetes de 100 pulsos a 4 kHz mientras se mantiene apretado. Ejemplo, X adelante:

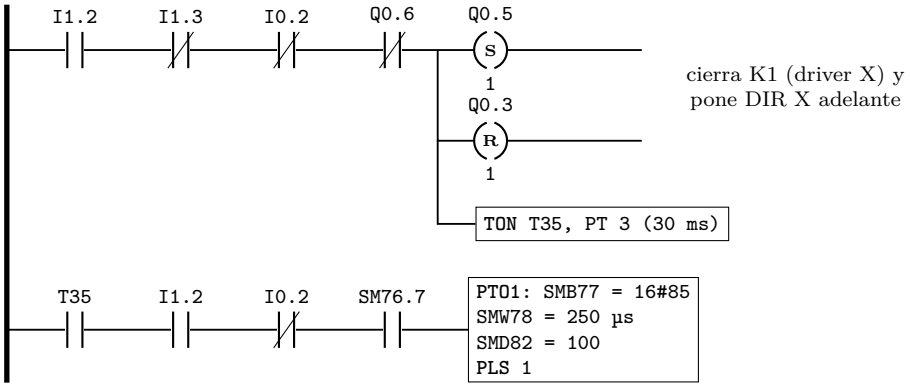


Figura 9: Jog de X adelante (SBR20, segmento 5). I0.2 impide pasar el final de carrera; Q0.6 abierto asegura que el tren no llegue también al driver Y.

Los segmentos 2–8 repiten este patrón para la base (I1.0/I1.1, sin relé), X atrás (I1.3) e Y (I1.4/I1.5 con K2). El segmento 9 abre los relés 30 ms después de soltar todos los pulsadores. El segmento 1 conmuta la garra con cada flanco de inicio.

4.3. Homing (SBR0)

El homing lleva cada eje a su sensor de origen, en este orden: X atrás, base al inductivo, Y abajo, abre la garra y pone las posiciones en cero.

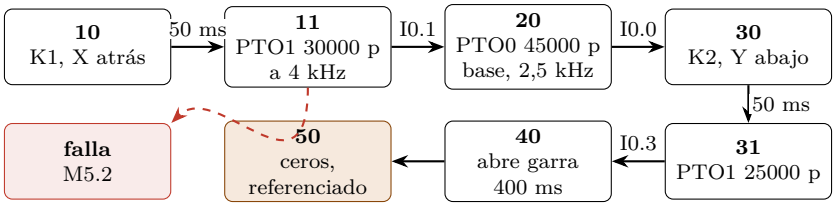


Figura 10: Máquina de pasos del homing (VW322). Cualquier paso de búsqueda puede terminar en falla.

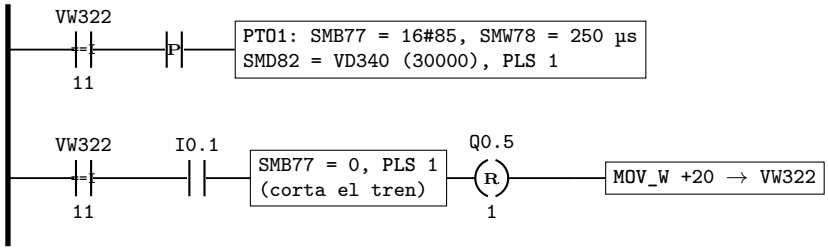


Figura 11: Pasos 11 a 20 del homing: arranca el tren hacia atrás y lo corta al llegar a I0.1.

Detección de fallas (segmento 9): el homing se declara fallido (M5.2, V200.3) si un paso de búsqueda dura demasiado (12 s en X e Y, 25 s en la base), si el tren de pulsos termina sin que aparezca el sensor, o si se pisa el final de carrera opuesto (flancos M5.5 de I0.2 y M5.6 de I0.4). En ese caso se cortan los trenes, se abren los relés y hay que rearmar.

4.4. Secuencias automáticas (SBR21)

Con referencia, cada inicio alterna entre la secuencia 1 y la 2. Son movimientos fijos en pulsos:

Paso	Eje	Movimiento	Pulsos
Secuencia 1 (pasos 10–40)			
10–12	Y	sube (K2, Q0.4 = 0)	12 000 (60 mm)
20–21	base	gira (Q0.2 = 0)	17 000
30–32	X	avanza (K1, Q0.3 = 0)	VD330 = 20 000 (100 mm)
40	garra	abre	–
Secuencia 2 (pasos 100–200)			
100	garra	cierra (toma la pieza)	–
110–112	Y	sube	5 000
120–122	X	retrocede (Q0.3 = 1)	VD334 = 14 000
130–131	base	gira (Q0.2 = 0)	18 000
140–142	Y	baja (Q0.4 = 1)	14 000
150	garra	abre (deja la pieza)	–
160–162	Y	sube	14 000
170–171	base	vuelve (Q0.2 = 1)	18 000
180–182	X	avanza	14 000
190–192	Y	baja	5 000
200	garra	abre; fin	–

Cada movimiento sigue el mismo patrón de tres pasos: cerrar el relé y fijar la dirección (esperar 20 ms), arrancar el tren de pulsos, y esperar a que SM76.7 (o SM66.7) indique que terminó.

4.5. Estado para la PC y limitaciones

El segmento 8 del OB1 publica el estado en VB200 (listo, ocupado, falla de homing, referenciado) y los pasos en VW320/VW322. Las constantes VD340 = 30000 y VD344 = 25000 sirven a la app como *firma* para reconocer este programa.

Limitaciones del programa original. No guarda la posición (solo sabe si hizo homing); solo hace dos rutinas fijas; para cambiarlas hay que reprogramar el PLC; el jog corta el paquete al soltar el pulsador, por lo que no se puede contar con exactitud. Por eso, con este programa, la app *estima* la posición (sección 11.4).

5. Programa del gemelo: `brazo_gemelo`

El programa `plc/brazo_gemelo.awl` se escribió para que el PLC sea la «mitad física» del gemelo. Conserva el modo manual y el homing del programa de Miguel, y agrega:

1. **cuenta de pulsos** de cada eje con signo: la posición es exacta;
2. **finales de carrera que cortan** el tren de pulsos y corrigen la cuenta;
3. un **buzón de órdenes** en la memoria V para que la PC pida homing, movimientos libres, garra y paro;
4. un **watchdog**: si la PC deja de dar señales de vida durante un movimiento, el PLC se detiene.

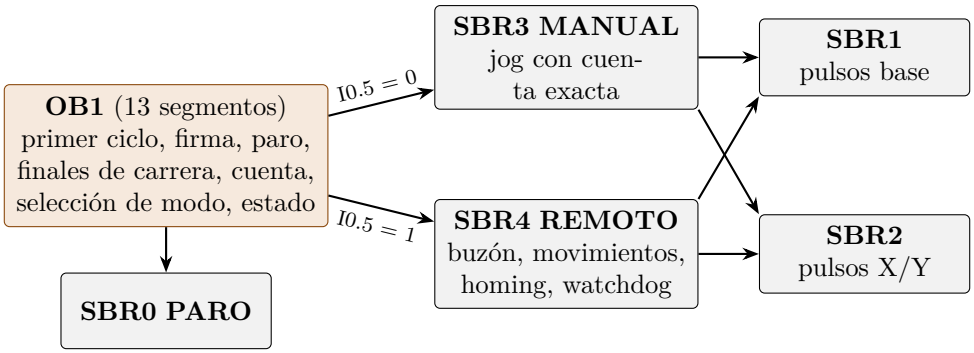


Figura 12: Estructura de `brazo_gemelo`. SBR1 y SBR2 son los únicos lugares donde se arrancan trenes de pulsos: así toda la cuenta pasa por un solo sitio.

5.1. Cuenta de pulsos

Cada vez que se arranca un tren, SBR1 o SBR2 guardan el *tramo* con su signo (según la DIR y los relés). Cuando el tren termina, el OB1 suma el tramo a la posición:

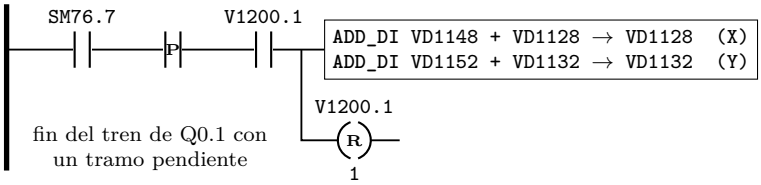


Figura 13: OB1, segmento 9: al terminar un tramo de Q0.1, se suma a X y/o a Y.

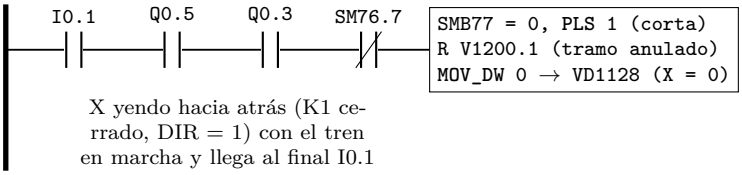


Figura 14: OB1, segmento 4: el final de carrera corta el tren y fija la posición exacta. Los segmentos 5–7 hacen lo mismo con I0.2 (X = 34000), I0.3 (Y = 0) e I0.4 (Y = 34000).

5.2. El buzón de órdenes

La PC escribe en la zona VB1100–VB1119 y el PLC publica su estado en VB1120–VB1159:

Dirección	Quién	Contenido
VB1100	PC	orden: 1 homing, 2 mover, 3 paro, 4 cerrar garra, 5 abrir garra
VB1101	PC	número de secuencia: el PLC ejecuta cuando cambia
VW1102	PC	latido (watchdog de 2 s)
VD1104/1108/1112	PC	pulsos relativos de base / X / Y, con signo
VW1116/1118	PC	ciclo del tren X/Y y de la base [µs]
VB1120	PLC	estado: 0 reposo, 1 homing, 2 moviendo
VB1121	PLC	eco de la secuencia
VB1122	PLC	bits: referenciado, remoto, PTO ocupados, posición incierta, watchdog
VB1123	PLC	error: 1 ocupado, 2 fuera de recorrido, 3 watchdog, 4 sin homing, 5 orden desconocida, 6 paro
VD1124/1128/1132	PLC	posición de base / X / Y en pulsos
VB1138–1140	PLC	copia de IB0, IB1 y QB0
VB1141	PLC	firma `16#47` (la app reconoce así el programa)

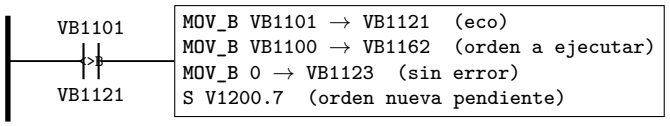


Figura 15: SBR4, segmento 8: una orden solo se toma cuando cambia el número de secuencia. Repetir la escritura no la ejecuta dos veces.

Luego, los segmentos 9–19 validan la orden pendiente, en este orden de prioridad:

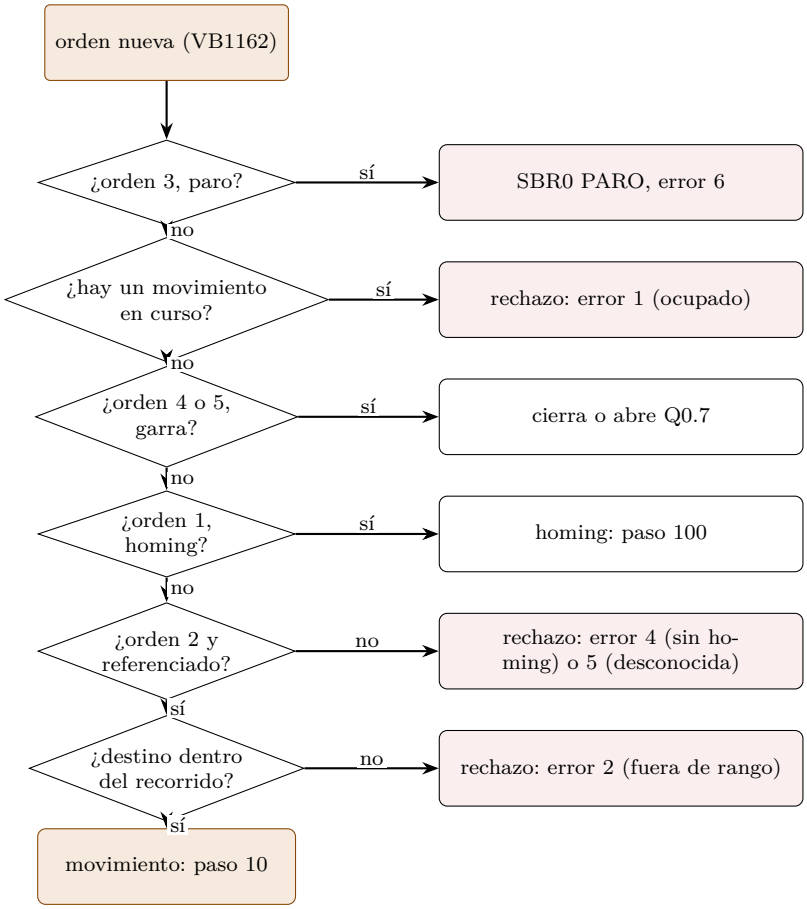


Figura 16: Validación de una orden en el PLC (SBR4, segmentos 9–19). El destino se calcula como posición actual + pulsos relativos y se rechaza si sale de 0–34000 en X/Y o de ±50000 en la base.

5.3. Ejecución de un movimiento

Un movimiento aceptado recorre los pasos 10–15 de VW1136. Resuelve el tren compartido calculando el tramo común y el resto:

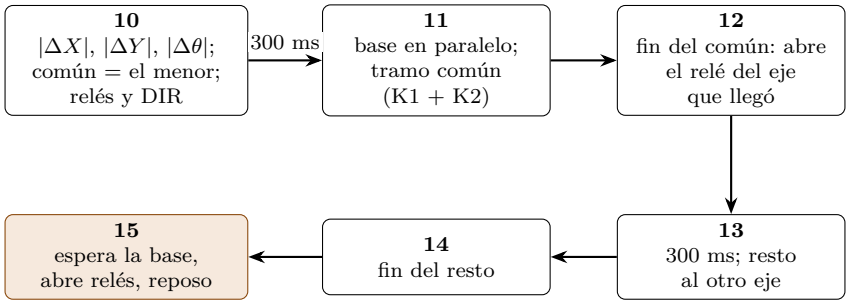


Figura 17: Máquina de pasos de un movimiento (SBR4, segmentos 20–51). Si no hay tramo común, el paso 11 salta al 13; si no hay resto, el 13 salta al 15.

El homing del programa del gemelo (pasos 100–109) sigue el orden del programa de Miguel: Y abajo hasta I0.3, X atrás hasta I0.1, abre la garra y gira la base hasta el inductivo I0.0; al final pone las tres posiciones en cero y marca *referenciado*.

5.4. Watchdog y paro

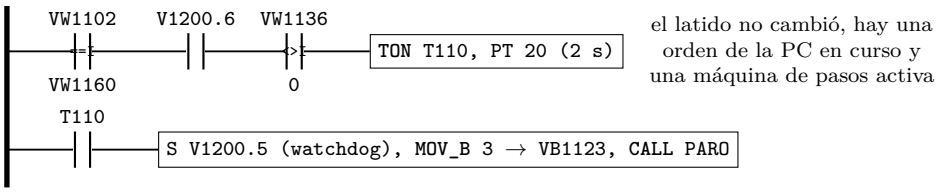


Figura 18: SBR4, segmentos 5–7: si la PC no cambia el latido en 2 s durante un movimiento, el PLC se detiene.

La subrutina PARO (SBR0) corta los dos trenes, abre los relés y vuelve al reposo. Si había un tramo en curso, marca la posición como *incierta* y pide un nuevo homing: el PLC no sabe cuántos pulsos alcanzaron a salir. Se llama desde el pulsador I0.6, desde un cambio del selector I0.5, desde la orden 3 de la PC y desde el watchdog.

5.5. Comparación de los dos programas

	Original	**brazo_gemelo**
Posición	no la conoce	contada al pulso
Movimientos	2 rutinas fijas	a cualquier punto del recorrido
Finales de carrera	solo en homing y jog	cortan cualquier movimiento
Mando desde la PC	iniciar, paro, emergencia	homing, mover, garra, paro, velocidad
Seguridad ante la PC	borra órdenes a los 5 s	watchdog de 2 s durante el movimiento
Velocidad	fija (4 kHz)	elegible (500–5000 Hz)

Parte III
El modelado

6. Levantamiento de medidas

El modelo parte de un levantamiento de la máquina: bocetos acotados de las piezas, fotos y el manual del fabricante. Cada cota quedó clasificada para saber qué tan confiable es:

Origen	**Ejemplo**	**Confianza**
leída (boceto o pieza)	guías de 300 mm, placas de 13 mm	alta
estimada (foto o manual)	longitudes del balancín y de la biela	media: se ajustó con fotos (< 5 mm)
SUPUESTO	paso del husillo, sentido de la base	por verificar en la planta

Todas las cotas están en un solo lugar, `web/js/cinematica.js`, que usan la página, la app, el modelo 3D y los scripts de gráficas.

7. Cinemática

7.1. El mecanismo

El brazo es un mecanismo de bielas en un plano vertical que gira con la base. El carro X desplaza todo el bastidor (cambia el alcance). El carro Y mueve un bloque; una biela de longitud L_{12} une ese

bloque con el balancín A–B, que es solidario al brazo: al moverse el bloque, el brazo gira un ángulo φ y la garra sube o baja.

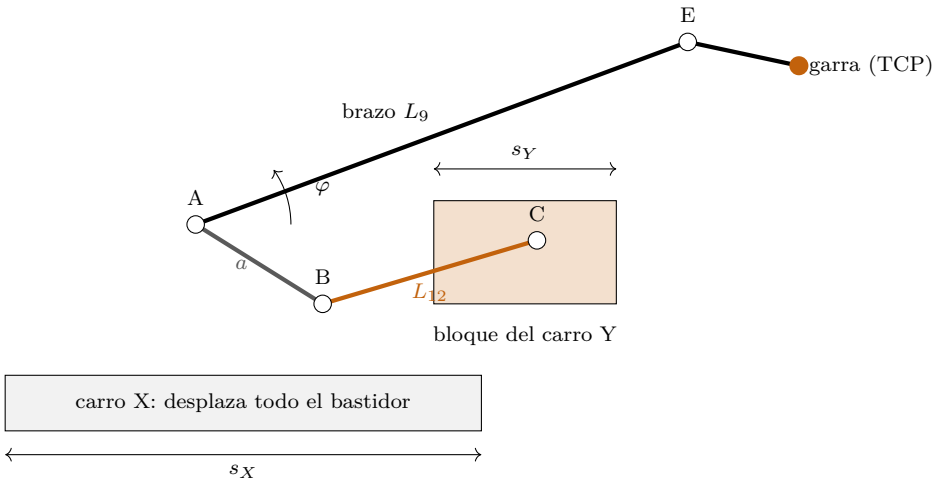


Figura 19: Esquema del mecanismo (no a escala).

7.2. Cinemática directa: de los carros a la garra

Dados $q = (\theta, s_X, s_Y)$:

1. La posición del bloque fija el punto C; el lazo A–B–C se cierra intersecando la circunferencia de radio a centrada en A con la de radio L_{12} centrada en C. Se elige la solución del montaje real (B por debajo de la recta A–C). Así se obtiene el ángulo ψ del balancín y el del brazo, $\varphi = \psi - \beta$.
2. El pivote A se mueve con el carro X: $u_A = u_0 + s_X$.
3. La garra queda en

$$r = u_A + L_9 \cos\varphi + d_g, \qquad z = h_A + L_9 \sin\varphi - h_g,$$

$$X = r\cos\theta, \qquad Y = r\sin\theta, \qquad Z = z.$$

7.3. Cinemática inversa: de la garra a los carros

Dado el punto (X, Y, Z):

$$\theta = \operatorname{atan2}(Y, X), \quad r = \sqrt{X^2 + Y^2}, \quad \sin\varphi = \frac{Z + h_g - h_A}{L_9}, \quad s_X = r - d_g - L_9\cos\varphi - u_0.$$

Con $\psi = \varphi + \beta$ se conoce B, y el bloque del carro Y debe estar donde la biela L_{12} alcanza a B: intersección de una circunferencia con la recta por la que se mueve C; de ahí sale s_Y. Se descartan las soluciones fuera del recorrido o que choquen con la mesa. Hay dos ramas posibles del brazo (hacia adelante y hacia atrás); la planta usa la primera.

Listing 1: Fragmento de la cinemática inversa (`web/js/cinematica.js`).

```
const s = (obj.Z + p.hg - d.hA) / p.L9;          // seno del ángulo del brazo
if (Math.abs(s) > 1) return { valida: false, motivo: 'Altura inalcanzable' };
for (const [rama, phi] of [['adelante', Math.asin(s)], ['atrás', Math.PI - Math.asin(s)]]) {
  const sX = r - p.dg - p.L9 * Math.cos(phi) - d.u5_0 - p.uA;   // alcance -> carro X
  const psi = phi + beta;                                        // balancín A-B
  const bx = p.a * Math.cos(psi), bz = p.a * Math.sin(psi);
```

```
  const disc = p.L12 * p.L12 - (d.ch - bz) * (d.ch - bz);          // biela L12 hasta C
  if (disc < 0) continue;
  // ... dos posiciones posibles del bloque; se queda la del montaje real
}
```

7.4. Jacobiano y singularidades

El jacobiano $J = \partial(X, Y, Z)/\partial(\theta, s_X, s_Y)$ dice cuánto se mueve la garra por cada milímetro de carro. Su determinante es $\det J = r \cdot \partial z/\partial s_Y$. Cerca de $\det J = 0$ (biela casi alineada con el balancín) un movimiento grande del carro Y casi no sube la garra: son posturas a evitar. La app muestra $\det J$ y el *ángulo de transmisión* (ideal 90°) en la telemetría.

7.5. Espacio de trabajo

Como s_X y s_Y van de 0 a 170 mm, la garra solo puede estar en la imagen de ese cuadrado, que en el plano vertical (r, z) es una franja curva:

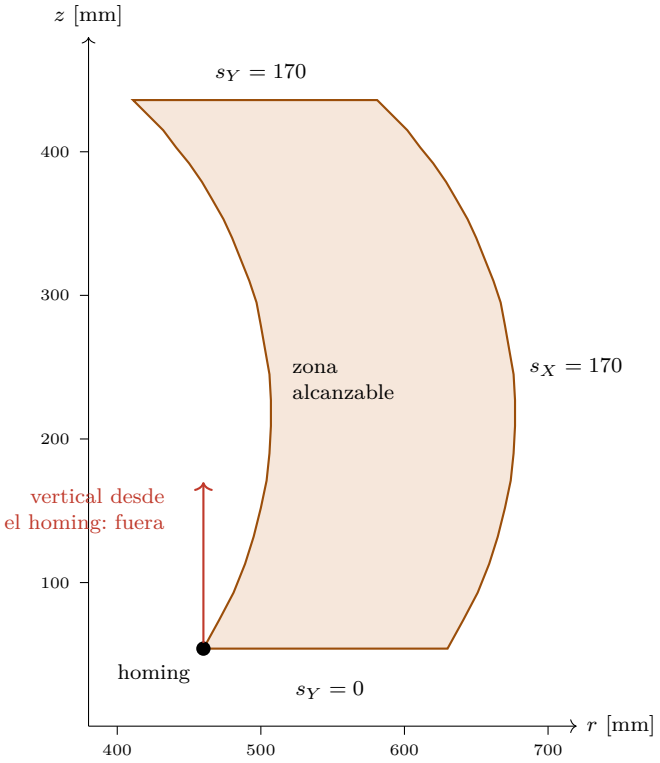


Figura 20: Corte vertical del espacio de trabajo, calculado con el modelo (θ fijo).

7.6. Conversión a pulsos

$$n_X = 200\, s_X, \qquad n_Y = 200\, s_Y, \qquad n_\theta = 190{,}3\, \theta \quad (\theta \text{ en grados}).$$

La app manda al PLC la diferencia entre el destino y la posición contada: $\Delta n = n_{destino} - n_{actual}$.

8. Modelo 3D

Las piezas del brazo se modelaron y se ensamblaron con las mismas cotas del modelo cinemático en un archivo `brazo_xk-js1-1.glb`. El visor (biblioteca three.js) carga el archivo y, en cada cuadro, coloca cada pieza según la cinemática directa: la base gira θ, el bastidor se desplaza s_X, el bloque s_Y, y la biela, el balancín y el brazo toman los ángulos del lazo. Así el dibujo nunca se separa del cálculo.

Parte IV
El software

9. Arquitectura

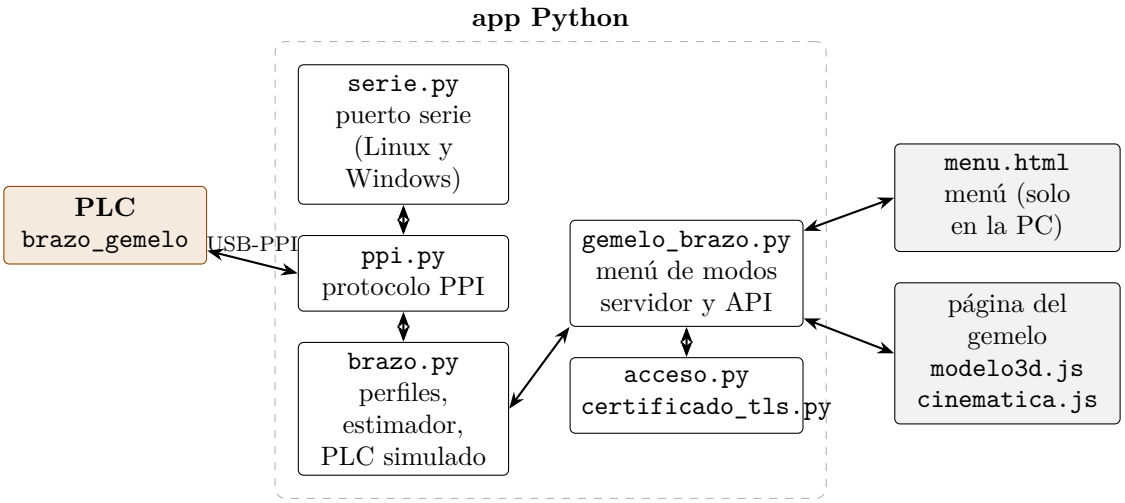


Figura 21: Módulos de la app y de la página.

Archivo	Responsabilidad
`serie.py`	Abre el puerto serie a 8 bits, paridad par y 1 parada: termios en Linux, API Win32 en Windows. Lista los puertos (USB primero).
`ppi.py`	Cliente PPI: tramas, acuses, sondeo, lecturas y escrituras S7; solo escribe en zonas permitidas; búsqueda de dirección y velocidad.
`brazo.py`	*Perfiles* según el programa del PLC (gemelo, cargado, monitor); bucle de lectura; estimador; PLC simulado; quién tiene el mando; registro.
`gemelo_brazo.py`	Menú de modos, cambio de modo en caliente, servidor local y de red, sesiones, filtros de órdenes, usuarios.
`acceso.py`, `usuarios.py`	Contraseñas con PBKDF2, sesiones de 8 h, bloqueo por intentos; administración por terminal.
`certificado_tls.py`	Certificado HTTPS autofirmado generado en Python (RSA 2048, SHA-256).
`ventana.py`	Ventana propia: GTK + WebKit en Linux, Edge en modo aplicación en Windows.
`probar_plc.py`	Diagnóstico de solo lectura del cable y del PLC.
`web/js/cinematica.js`	Modelo matemático (sección anterior).
`web/js/modelo3d.js`	Visor 3D, pestañas de mando, simulación en la página, seguimiento del brazo real.
`web/js/conexion-plc.js`	Llamadas a la API de la app.

10. La app: modos y cambio de modo

La app empieza sin conexión y muestra el menú. Al elegir un modo, `aplicar_modo` crea la conexión que corresponde y abre o cierra el acceso por red:

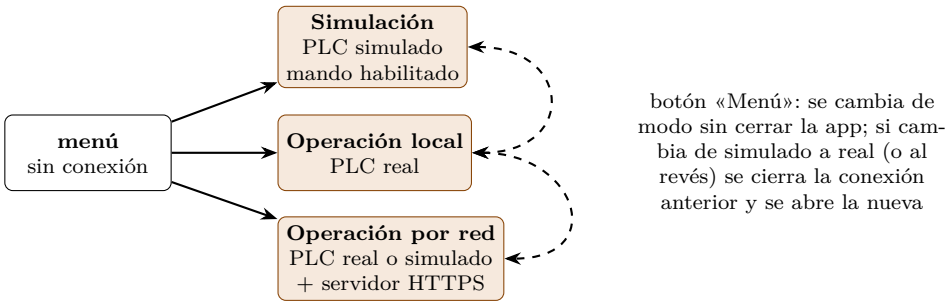


Figura 22: Modos de la app.

Listing 2: Cambio de modo (`gemelo_brazo.py`).

```
def aplicar_modo(modo, red_simulado=False):
    if modo == 'red' and not ACCESO.hay_usuarios():
        raise ValueError('para la operación por red cree antes al menos un usuario')
    simulado = modo == 'simulacion' or (modo == 'red' and red_simulado)
    with CAMBIO:
        if BRAZO is None or BRAZO.simulado != simulado:
            if BRAZO:
                BRAZO.cerrar()                  # detiene el brazo y suelta el puerto
            BRAZO = Brazo(Config.puerto, Config.direccion, simulado, REGISTRO)
        if modo == 'red':
            _activar_red()                      # servidor HTTPS en 0.0.0.0:8443
        else:
            _desactivar_red()
```

11. Comunicación con el PLC

11.1. El puerto serie en dos sistemas

El cable USB-PPI es, para la PC, un puerto serie. `serie.py` ofrece la misma interfaz (`abrir`, `escribir`, `leer`, `vaciar_entrada`) sobre las funciones de cada sistema:

Listing 3: Configuración del puerto en Windows por ctypes (`serie.py`).

```
dcb.BaudRate = self.baudios
dcb.ByteSize, dcb.Parity, dcb.StopBits = 8, 2, 0        # 8 bits, paridad par, 1 parada
dcb.flags = 0x0001 | 0x0002 | (1 << 4) | (1 << 12)      # binario, paridad, DTR y RTS
_k32.SetCommState(h, ctypes.byref(dcb))
t = _TIEMPOS(_MAXDWORD, _MAXDWORD, 20, 0, 2000)          # leer: lo que haya, o esperar 20 ms
_k32.SetCommTimeouts(h, ctypes.byref(t))
```

11.2. El protocolo PPI

Cada lectura o escritura es un intercambio de cuatro tramas a 9,6 kbit/s: la PC envía la petición (trama SD2), el PLC acusa recibo (`E5`), la PC pregunta por la respuesta (trama SD1) y el PLC la entrega (SD2 con los datos). El detalle byte a byte está en el documento de arquitectura.

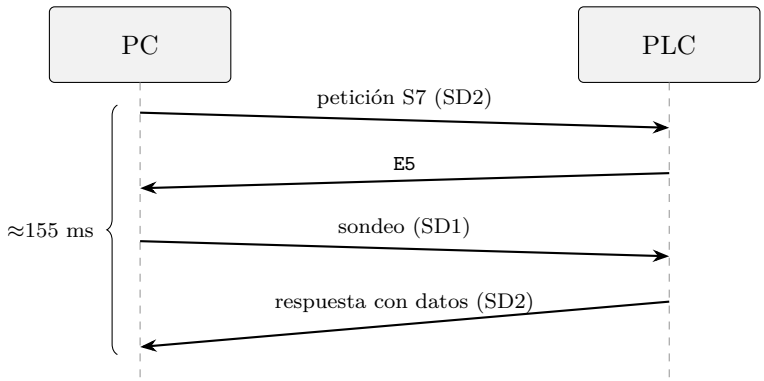


Figura 23: Intercambio PPI.

Listing 4: Núcleo del intercambio (`ppi.py`).

```
def _intercambio(self, pdu):
    self.p.vaciar_entrada()
    self._pausa()                              # silencio de ~150 bits: el PLC lo necesita
    self.p.escribir(self._sd2(0x7C if self._fcb else 0x6C, pdu))
    self._fcb = not self._fcb
    if not self._leer(0.4).startswith(b'\xe5'):
        raise ErrorPPI('el␣PLC␣no␣acusó␣la␣petición')
    for intento in range(8):                   # sondeo hasta que llegue la respuesta
        self._pausa()
        self.p.escribir(self._sd1(0x5C if intento % 2 == 0 else 0x7C))
        r = self._leer(0.4)
        if r.startswith(b'\x68'):
            return r[7:4 + r[1]]               # mensaje S7 de la respuesta
```

11.3. Perfiles: la app se adapta al programa del PLC

Al conectar, la app lee la firma: si VB1141 = `16#47` es `brazo_gemelo` (perfil *gemelo*); si VD340/VD344 = 30000/25000 es el programa original (perfil *cargado*); si no, solo monitorea. Cada perfil sabe leer el estado y traducir órdenes:

Listing 5: Envío de una orden al buzón del programa del gemelo (`brazo.py`).

```
def _enviar(self, orden, dz=0, dx=0, dy=0):
    seq = (self.ack + 1) & 0xFF                # el PLC ejecuta cuando cambia
    self.hb = (self.hb + 1) & 0xFFFF           # latido
    bloque = struct.pack('>BBHiiiHH', orden, seq, self.hb, dz, dx, dy, self.ciclo_xy, self.ciclo_z)
    self.cli.escribir('V', 1100, bloque)       # VB1100..VB1119 en una sola escritura
```

11.4. El estimador (programa original)

Con el programa original el PLC no cuenta la posición; la app la estima leyendo, unas 6 veces por segundo, qué trenes de pulsos están activos, sus ciclos, las direcciones y los relés:

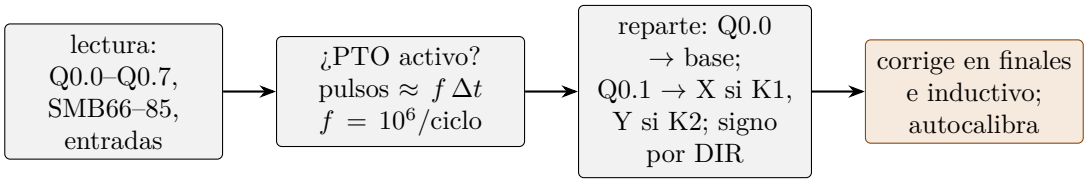


Figura 24: Estimación de la posición con el programa original.

Los tramos largos (homing, secuencias) se ajustan al número exacto de pulsos al terminar; el jog del panel se integra con un factor de trabajo (0,93, SUPUESTO) que se autocalibra cada vez que un carro va de un final de carrera al otro. En las pruebas simuladas el error máximo fue de 1,4 mm.

12. La página y los modos de mando

Modo	Cómo funciona	Limitaciones
Articular	se fijan θ, s_X, s_Y; la directa calcula la garra; se envía el destino	X e Y a la misma velocidad; sin rampas
Cartesiano	se fija (X, Y, Z); la inversa calcula los carros; en simulación se sigue una recta con perfil suave $w = t^2(3 - 2t)$	con el brazo real la trayectoria no es recta; franja de trabajo estrecha; cotas estimadas
Secuencias	se graban puntos (pose + garra) y se reproducen esperando el reposo del PLC; se exportan como CSV de pulsos	los puntos se guardan en el navegador; sin condiciones ni velocidades por punto

El visor sigue al brazo real *interpolando* entre las dos últimas lecturas con un retardo de 0,1–0,5 s, para que el movimiento se vea continuo aunque las lecturas lleguen cada 150 ms.

13. Seguridad por capas

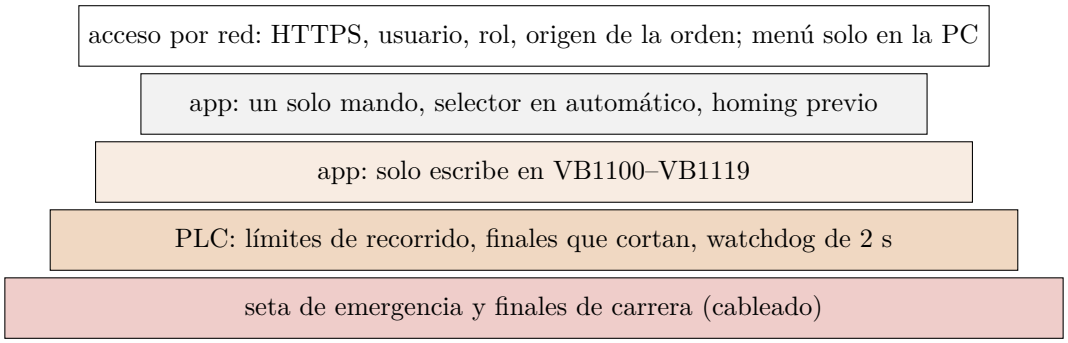


Figura 25: Capas de seguridad, de la más cercana a la máquina (abajo) a la más externa.

Parte V
Referencia

14. Archivos del proyecto

```
Brazo-4GDL-gemelo-digital/
  index.html, web/js/, web/css/        página publicada (simulación y visor)
  global/modelo-3d/brazo_xk-js1-1.glb  modelo 3D
  semestres/2026_II/
    modelo-3d/                         levantamiento, cinemática, gráficas
    gemelo-digital/
      app/                             app unificada (Linux y Windows)
      plc/brazo_gemelo.awl, .mwp       programa del gemelo
      manual/                          manuales, arquitectura y esta guía
```

```
ETN 1034/respaldo_vm/                  programas originales respaldados
```

15. Glosario

AWL	lista de instrucciones del S7-200; el formato de texto de los programas.
Gemelo digital	modelo en la computadora que reproduce el estado del equipo real en tiempo real.
Homing	búsqueda de los sensores de origen para fijar el cero de cada eje.
Jacobiano	matriz de derivadas que relaciona velocidades de los ejes con la velocidad de la garra.
Jog	movimiento manual mientras se mantiene apretado un pulsador.
PPI	protocolo de comunicación del S7-200 por RS-485.
PTO	salida de tren de pulsos del PLC.
TCP	punto de trabajo de la herramienta: el centro de las mordazas.
Watchdog	vigilancia que detiene la máquina si deja de recibir señales de vida.